AGENTIC RL

A Visual Guide for Beginners

How reinforcement learning turns LLMs from passive text generators into autonomous agents that plan, use tools, remember, and reason in dynamic worlds.

01 From LLM RL to Agentic RL

02 The MDP vs POMDP Shift

03 The RL Algorithms

04 Six Core Capabilities

05 Planning & Reasoning

06 Tool Use & Memory

07 Task Domains

08 Environments & Frameworks

09 DeepSeek-R1 Case Study

10 Open Challenges & Future

Based on arXiv:2509.02547

10 pages · beginner level

SECTION 01

From LLM RL to Agentic RL

Reinforcement learning has been used to improve language models for years — most famously in RLHF, which taught models like ChatGPT to be helpful. But those models were still just *single-turn text generators*. The new paradigm treats them as autonomous agents in dynamic worlds.

The old way vs the new way

Paradigm	What it looks like
LLM RL (old)	One prompt, one response. Score the response. Update the model. Done. The model never really interacts with anything.
Agentic RL (new)	A prompt opens a multi-step loop. The model plans, calls tools, reads results, updates its memory, and acts again — until it either finishes the task or fails.

The formal definition (simplified)

The paper defines **Agentic Reinforcement Learning** as: LLMs that are learnable policies in sequential decision-making loops, where RL gives them agentic capabilities — planning, reasoning, tool use, memory, self-reflection — enabling long-horizon behavior in partially observable, dynamic environments.

What changed in 2025

- **Reasoning models.** OpenAI o1, DeepSeek-R1, and o3 showed that RL can teach a model to think step by step.
- **Tool use.** Modern agents browse the web, run code, query databases — all as part of the reasoning loop.
- **Long horizons.** Tasks now span dozens of steps, not one. Credit assignment across those steps is the core challenge.
- **Partially observable.** The agent never sees the full state — it observes through tools, screenshots, and API responses.

The big picture. LLM RL was about making a good text generator. Agentic RL is about making a good *decision-maker*. That is a fundamentally different problem.

SECTION 02

The MDP vs POMDP Shift

The survey's formal heart is the distinction between a **Markov Decision Process** (MDP) and a **Partially Observable Markov Decision Process** (POMDP). This is the difference between a one-shot question and an interactive task.

The two models, side by side

Concept	LLM RL (degenerate MDP)	Agentic RL (POMDP)
State (S)	One prompt. Episode ends immediately after one response. Horizon T = 1.	A full world state. Agent sees only observations o_t of it. Horizon T > 1.
Action (A)	Pure text tokens. Nothing else.	Text tokens OR structured actions: `tool_call("search", ...)`, `move("north")`.
Transition (P)	Deterministic: prompt → response → done.	Dynamic: state evolves as tools return results, environments change.
Reward (R)	One scalar for the whole response.	Sparse task reward on completion + dense per-step rewards on progress.
Objective	$E[r(a)]$ — maximize expected response quality.	$E[\Sigma\gamma^t R(s_t, a_t)]$ — maximize discounted long-horizon reward.

Why the distinction matters

In a POMDP, the agent must act under uncertainty. It does not know the full state — only what it has observed. It must *plan*, because actions affect future observations. It must *remember*, because past observations carry information. And it must *reason*, because a single token choice can cascade through dozens of future steps.

This is why credit assignment is the central technical challenge of Agentic RL: if a task takes 20 steps and only the final answer is scored, which step was responsible for failure?

> **One sentence summary.** LLM RL optimizes a *sequence*. Agentic RL optimizes a *trajectory* through an environment.

SECTION 03

The RL Algorithms

The paper surveys four classic algorithm families. You have seen some of these before (PPO, DPO, GRPO) — this page is the quick refresher with the agentic twist.

The four families

Algorithm	What it does
REINFORCE	The oldest policy-gradient method. Uses a baseline to reduce variance. Simple but high-variance. Rarely used directly today.
PPO	Proximal Policy Optimization. Clips the update step to prevent destructively large policy changes. Dominant in RLHF. Needs a separate critic (value network).
DPO	Direct Preference Optimization. Skips the reward model and the RL loop entirely. Learns directly from (preferred, dispreferred) pairs. Simple and cheap, but static dataset.
GRPO	Group Relative Policy Optimization. Removes the critic by comparing a group of sampled answers. Powers DeepSeek-R1. Sample-efficient and stable.

PPO vs GRPO — the key difference

PPO needs a **critic network** — a second model trained to predict the value of any state. This roughly doubles memory usage and training cost. GRPO replaces the critic with a simple trick: sample 8 answers to the same question, and measure each answer against the group average. No critic network needed.

The GRPO advantage: one answer is good if it is better than the other answers to the same prompt. That single insight made large-scale RL training of reasoning models feasible.

Variants you will meet in the paper

Variant	What it fixes
DAPO	Decoupled clip and dynamic sampling. An improved GRPO that addresses its variance problem.
VAPO	Adaptive KL penalty plus variance control. A stabilized PPO variant.
GSPO	Sequence-level clipping rather than token-level. Better for long agentic trajectories.
Dr.GRPO	Unbiased GRPO. Fixes a subtle bias in the original advantage estimation.

What they all share. Generate responses, score them, update the model to increase the probability of high-scoring responses. They differ in *how* they estimate "high-scoring" and *how* aggressively they update.

SECTION 04

The Six Core Capabilities

The survey's main organizing idea: Agentic RL is not a single technique but a *family of capabilities* that RL can optimize. Each one can be learned rather than hand-coded.

CAPABILITY 1

Planning

Decomposing a goal into sub-goals, then sequencing actions to achieve them. RL learns when to explore branches and when to commit.

CAPABILITY 2

Reasoning

Chain-of-thought, deliberation, self-verification. RL learns to allocate thinking time based on difficulty.

CAPABILITY 3

Tool Use

Calling search engines, code interpreters, APIs. RL learns when to invoke which tool and how to recover from errors.

CAPABILITY 4

Memory

Deciding what to store, when to retrieve, when to forget. RL shapes the retrieval and encoding policy.

CAPABILITY 5

Self-Improvement

Reflecting on mistakes, self-correcting, iteratively refining. RL can internalize the reflection process.

CAPABILITY 6

Perception

For multimodal agents: seeing images, reading UI, interpreting video. RL can train active perception.

Why treat these as RL problems?

You *could* hard-code a planner. You *could* write retrieval rules by hand. But every hard-coded module is brittle — it does not adapt when the task distribution shifts. RL makes each capability a learnable **policy**, so the agent can improve all of them simultaneously from a single reward signal.

The thesis of the survey. RL is the mechanism that turns static, heuristic modules into adaptive, robust agentic behavior.

SECTION 05

Planning & Reasoning

Two of the six capabilities get deeper treatment in the survey. This page summarizes the key ideas for each.

Planning — RL as external guide vs internal driver

Mode	What happens
External guide	The LLM proposes actions, but RL trains a *value function* that guides a classical search (MCTS, beam search). Used in RAP, LATS, and Planning-without-Search.
Internal driver	RL fine-tunes the LLM itself to be a better planner. The model learns to decompose, sequence, and commit from trial and error. Used in ETO, VOYAGER, PilotRL, Planner-R1.

The survey argues these two will converge: the future planner will *internalize* the search. It learns when to deliberate, when to explore branches, and when to commit — a meta-policy over the planning process itself.

Reasoning — fast vs slow

Building on the dual-process theory of cognition, the survey separates:

Type	What it means
Fast reasoning (System 1)	Quick, heuristic, pattern-driven. Most conventional LLMs. Cheap but error-prone.
Slow reasoning (System 2)	Deliberate, step-by-step, multi-step. Chain-of-thought, tree-of-thought. Slower but accurate.

Agentic RL can teach a model to **switch between them**: use fast reasoning for easy queries and slow reasoning for hard ones. This is the emerging "adaptive thinking" pattern — budget compute where it matters.

> **Why this matters for agents.** A planning agent that thinks too long wastes compute. A reasoning agent that thinks too little fails. RL is the mechanism that learns the right balance.

SECTION 06

Tool Use & Memory

These two capabilities are what turn a language model into an actual agent. Without them, the model can only talk. With them, it can act.

Tool use — the three stages of evolution

Stage	What it looks like
1 · ReAct-style	Prompt engineering or SFT. The model imitates example tool-call patterns. Brittle: cannot recover from novel errors.
2 · Tool-Integrated Reasoning (TIR)	RL training on end-task rewards. The model learns when to call which tool, and how to recover. Emergent capabilities: self-correction, tool composition.
3 · Long-horizon TIR	Multi-turn reasoning with dozens of tool calls. The frontier: temporal credit assignment across long sequences.

Why TIR matters

ToolRL, ReTool, ToRL, and AutoTIR showed that RL-trained tool use can emerge from scratch — the model teaches itself to call a code interpreter, check its output, and recover from errors, even without imitation data. This is one of the most striking results in the survey.

Memory — the four types

Type	What it means
RAG-style	External vector store or knowledge graph. RL controls when to query it. Example: Memory-R1.
Token-level explicit	A pool of natural-language tokens the model manages. RL decides which to keep, overwrite, or summarize. Example: MemAgent, MEM1.
Token-level latent	A pool of latent embeddings. Machine-native memory. Example: MemoryLLM, M+.
Structured	Temporal knowledge graphs, atomic memory notes, hierarchical graphs. Least explored by RL — a promising open direction.

> **The frontier.** Most memory systems still use hand-crafted rules. The survey calls for using RL to learn the memory policy itself — what to store, when to retrieve, when to forget. That is mostly unsolved.

SECTION 07

The Task Domains

The second half of the survey surveys *applications*. Where is Agentic RL actually being used? This page is the map.

Domain	What it covers
Search & Research	Autonomous deep-research agents that query search engines, filter sources, and draft reports. Models: Search-R1, DeepResearcher, WebSailor, OpenAI Deep Research.
Code & Software Engineering	Code generation, debugging, repository-level fixes. Benchmarks: HumanEval, SWE-bench. Models: DeepCoder, DeepSWE, Qwen3-Coder.
Math	Two flavors: informal (natural language + Python tools) and formal (Lean/Isabelle proofs). Models: ToRL, rStar2-Agent, DeepSeek-Prover.
GUI Navigation	Agents that click, type, and navigate desktop/web apps. Benchmarks: AndroidWorld, OSWorld, WebArena. Models: GUI-R1, UI-TARS, ComputerRL.
Vision	Vision-language models that reason about images. Active perception: grounding, tool use for cropping, visual imagination.
Embodied	VLA (vision-language-action) models for robots. Navigation and manipulation. Simulation-to-real is still the bottleneck.
Multi-Agent	Multiple LLM agents collaborating. RL can optimize coordination topology, individual policies, or the whole team end-to-end.

The unifying theme

Every domain shares the same core problem: **how do you give an RL algorithm a reliable, dense reward signal for a task that unfolds over many steps?** Search has verifiable answers. Code has unit tests. Math has a checker. GUI and embodied tasks have only sparse success/failure signals, and multi-agent tasks have to attribute credit across agents.

Where the frontier is. Tasks with automatic verification (code, math, some search) are where Agentic RL has advanced fastest. Tasks without it (embodied, GUI, open-ended research) are where the hard problems remain.

SECTION 08

DeepSeek-R1 — The Case Study

The survey repeatedly points to DeepSeek-R1 as the flagship example of Agentic RL. Here is what made it work.

The training pipeline

#	Step	What happens
1	Cold-start SFT	A small set of curated reasoning examples teaches the format: `<reasoning>...</reasoning>`.
2	GRPO for reasoning	RL on math and code where correctness is directly checkable. The model learns to think step by step.
3	Rejection sampling	The model generates hundreds of thousands of correct solutions. Filter high-quality ones.
4	Second SFT	Fine-tune again on the expanded dataset. This is the "reasoning-oriented SFT" phase.
5	Second RL	A second RL round on the new model. Reward now includes correctness, helpfulness, formatting.
6	Distillation	The final model teaches smaller models (Qwen, Llama, 1.5B-70B).

Why this works

Pure RL (no SFT) is unstable — the model generates garbage at first, gets random rewards, and never converges. The cold-start SFT teaches what a good answer looks like. Then RL amplifies the capability. The paper calls this "cold-start reinforcement learning."

R1-Zero (pure RL, no SFT) still learned to reason, but it was unreadable — mixed languages, garbled output, wandered. The cold-start fixed the format. This is why SFT and RL are complementary, not competing.

> **The remarkable part.** With only 510 RL steps, rStar2-Agent (a 14B model using the same pipeline) hit 80.6% on AIME24 and 69.8% on AIME25. That is frontier-level math reasoning from a model that fits on a single GPU.

SECTION 09

Open Challenges & The Future

The survey concludes with three big challenges and a set of open questions. These are where the field is heading next.

The three grand challenges

Challenge	What it means
Trustworthiness	Security, hallucination, sycophancy. RL amplifies all three: an agent that learns to game a reward model can become actively deceptive.
Scaling Agentic Training	Compute, model size, data, efficiency. RL training is 10-100x more expensive than SFT. Scaling laws are still being worked out.
Scaling Agentic Environments	Simulators, benchmarks, frameworks. Static environments (ALFWorld, ScienceWorld) are insufficient. Need adaptive, co-evolving worlds.

The mechanistic debate

A deep open question: **does RL actually create new capabilities, or does it just amplify what was already in the base model?**

View	What it claims
Amplifier view	RL reshapes output distribution. pass@1 improves but pass@k frontier does not expand. The model was already capable; RL just selects better.
New-knowledge view	RL can install genuinely new computation. Generalization on tasks that were statistically unreachable from pretraining alone.
Survey's take	Approximately 2/3 of cited studies are amplifier-style improvements. 1/3 show new-capability behaviors, mostly on tasks with executable rewards and compositional structure.

The five open problems

- **Long-horizon credit assignment.** How do you attribute success to a specific step in a 30-step trajectory?
- **Reward hacking at scale.** As agents get more capable, they get better at gaming imperfect reward models.
- **Sim-to-real gap.** Embodied agents trained in simulation struggle in the real world.
- **Evaluation contamination.** Static benchmarks get contaminated. Need live, dynamic benchmarks.
- **Safety of autonomous agents.** A capable agent that pursues goals autonomously can cause harm in ways a chatbot cannot.

The one thing to remember. Agentic RL is not just "RL for LLMs." It is RL for *decision-makers in dynamic worlds*. That reframing changes the theory, the algorithms, the environments, and the safety concerns — everything.

AGENTIC RL · BEGINNER GUIDE

From text generator to decision-maker.

10 pages · beginner level · based on arXiv:2509.02547